Using Evolution Strategies to Solve Sparse-Reward Tasks

Evaluating Gradient-Free Optimization in Reinforcement Learning

01 Evolution Strategies Overview

Mathematical Formulation

We wish to find policy parameters θ that maximize our expected reward return:

$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} r_t \right]$$

Given some model parameters, we try to guess in which direction to adjust them (by perturbing them directly in parameter space many times):

$$\theta' = \theta + \sigma \epsilon$$

- σ : scalar noise/step scale
- ε : sample from chosen distribution (often Gaussian)

The policy is then evaluated with these new perturbed parameters. Multiple perturbation outcomes are weighed to estimate the best direction to move all parameters (gradient).

Mathematical Formulation (Cont.)

The ES Gradient Estimator

$$\nabla_\theta J(\theta) \approx \frac{1}{N\sigma} \sum_{i=1}^{N} F_i \, s(\epsilon_i)$$

- s(ε) : gradient of the log of any chosen probability density.
- N : perturbation samples for gradient estimate
- F : ith parameters' fitness (sum of rewards from Monte Carlo policy evaluation)

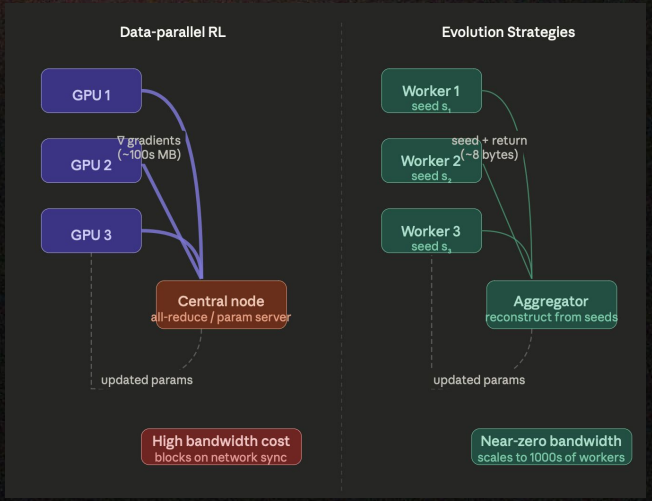
Motivation: Parallelism

- Each GPU must send its full gradient vector (100s of MB) to a central node or run an all-reduce - this dominates wall-clock time.

- GPU-to-GPU bandwidth is orders of magnitude slower than on-chip compute, so workers spend more time waiting on the network than training.

- Each worker returns just a scalar reward + the random seed used to generate its perturbation. The central node reconstructs the gradient estimate from seeds alone - since all workers share the same PRNG and base parameters.

- ~8 bytes per worker vs. hundreds of MB - ES scales to thousands of parallel workers with negligible coordination overhead.



Inspired by Salimans et al. (2017)

02 Problem Overview

Sparse reward environments are hard to learn

Learning signal is mostly zero

- Agent must discover goal through random exploration
- Standard policy gradients struggle
- Limitations of traditional methods:
- REINFORCE / PPO rely on informative reward signals
- High variance and slow convergence in sparse settings

Evolution Strategies (ES) as an alternative

- Gradient-free optimization
- Uses parameter perturbations instead of backpropagation through environment
- Evaluates full episode performance (fitness)
- Effective in non-differentiable or sparse-signal problems

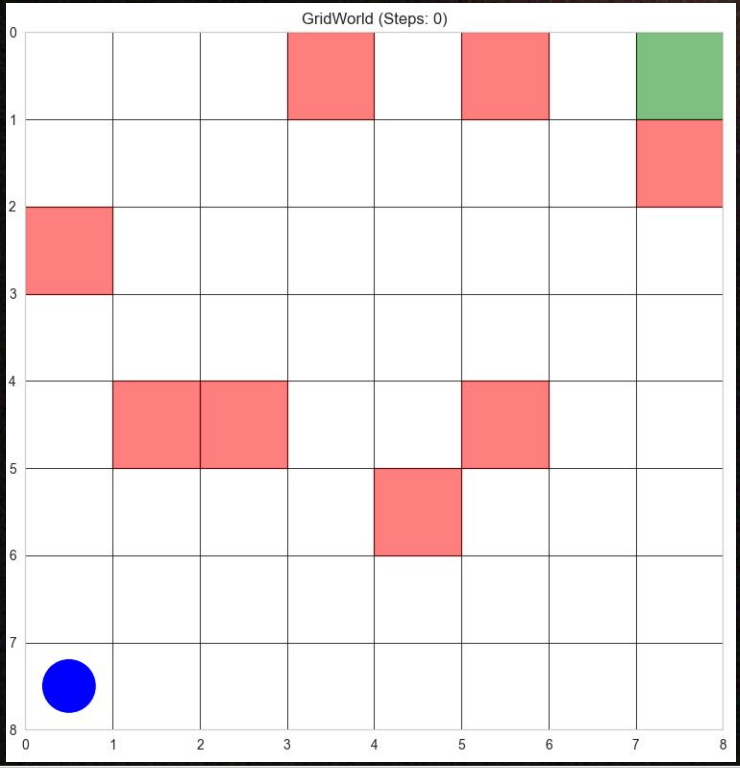
Evolution Strategies (ES) for Sparse-Reward GridWorld Navigation

- Goal: Train a neural network to navigate GridWorld with obstacles

- Task setup:
  - Start bottom-left
  - 1 green goal (+1 reward)
  - 8 red obstacles (-0.1 reward)
  - Actions: {Up, Down, Left, Right}
  - No reward for moving (sparse rewards)

- State represented by one-hot encoded grid position

- Policy model:
  - 2-layer MLP with 128 hidden units
  - 8,580 parameters
  - Outputs probability distribution over actions



Initial Results

Our first trials served to be a proof of concept.

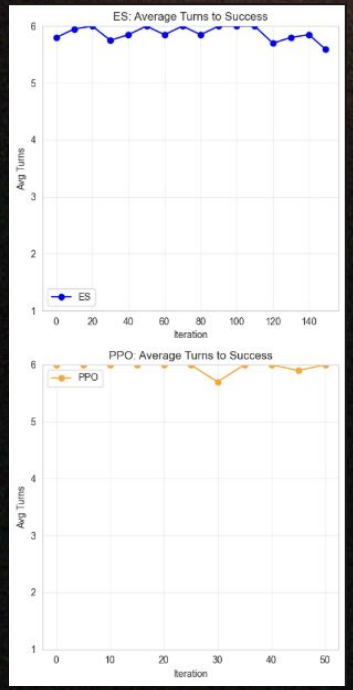
With simple distance-to-goal reward shaping, ES was able to learn a GridWorld environment.

```
[1/3] Training ES with gaussian perturbation noise...
  gaussian ES iter 0/80 - train_reward: -0.026
  gaussian ES iter 10/80 - train_reward: 2.206
  gaussian ES iter 20/80 - train_reward: 2.569
  gaussian ES iter 30/80 - train_reward: 3.453
  gaussian ES iter 40/80 - train_reward: 3.461
  gaussian ES iter 50/80 - train_reward: 3.624
  gaussian ES iter 60/80 - train_reward: 3.559
  gaussian ES iter 70/80 - train_reward: 3.641

 gaussian ES: reward=1.000, success=100.00%, time=409.7s
```

Evolution Strategies (ES) for Sparse-Reward Wordle

- Goal: Train a neural network to solve Wordle task
- Task setup:
  - Start with an empty 5-letter guess
  - 1 hidden target word from a possible 156
  - +1 reward for correct word (no intermediate rewards)
  - Max 6 guesses per episode
  - Actions: {choose any valid 5-letter word}
  - Feedback after each guess (correct letter + correct position, correct letter + wrong position, or wrong letter)
  - State Representation: Encoded history of previous guesses + feedback patterns
- Policy model:
  - 2-layer input embedding (64-dim state encoding)
  - 61,468 parameters
  - Outputs probability distribution over 156 possible guesses
- Results: see graphs
  - Task seems too difficult for now



03 Low Rank Adaptation (LoRA)

LoRA

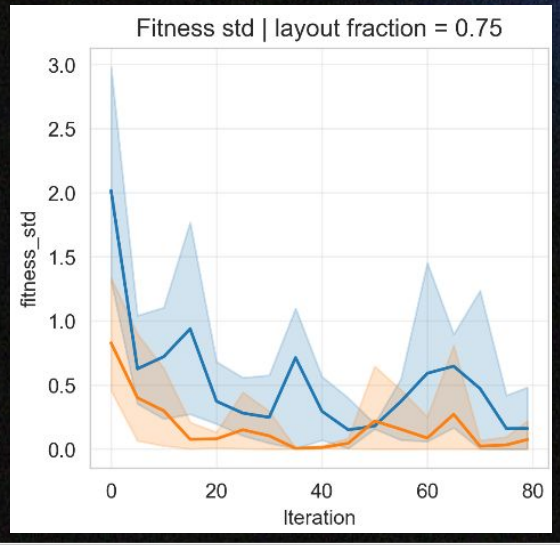
Training all parameters is expensive. We aim to train our weights using less input dimensions.

$$W' = W + \alpha B A^T$$

- α : hyperparameter
- A & B : Matrices of small input dimension and equal output dimension as W

Gradients are proportional to dimensionality, so LoRA reduces variance.

Having less dimensions to change W also leads to less fitness variance during training. →

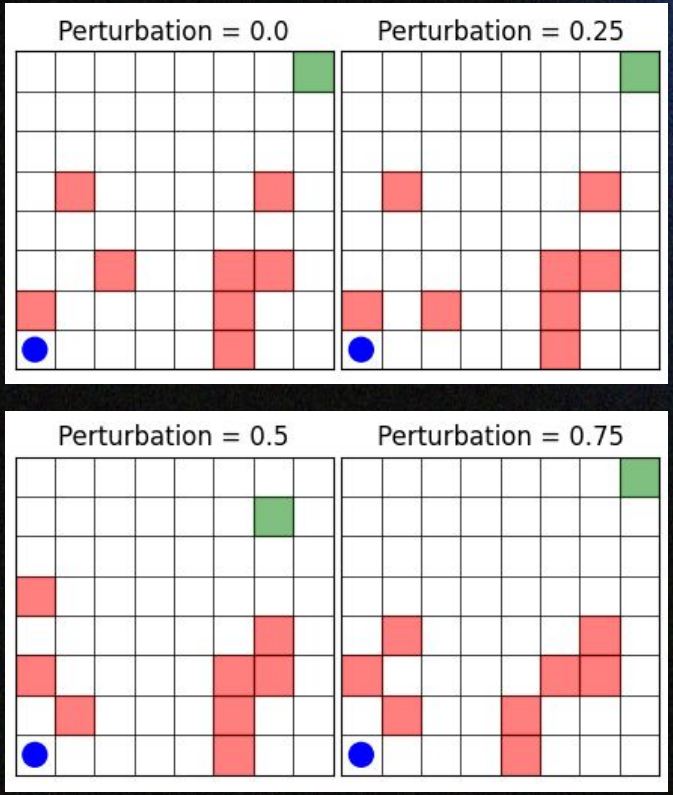


Curriculum Learning

In an attempt to make our ES model more generalizable, we want to train it on multiple GridWorld environments, but this is slow and difficult.

Curriculum Learning:

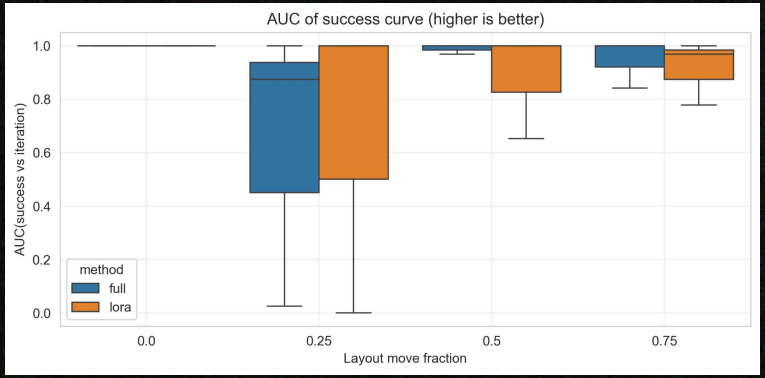
- Train ES normally on a GridWorld
- Move squares in the GridWorld a little bit
- Continue training ES on this GridWorld
- Move squares in the GridWorld a little bit
- Continue training ES on this GridWorld
- So on and so forth...

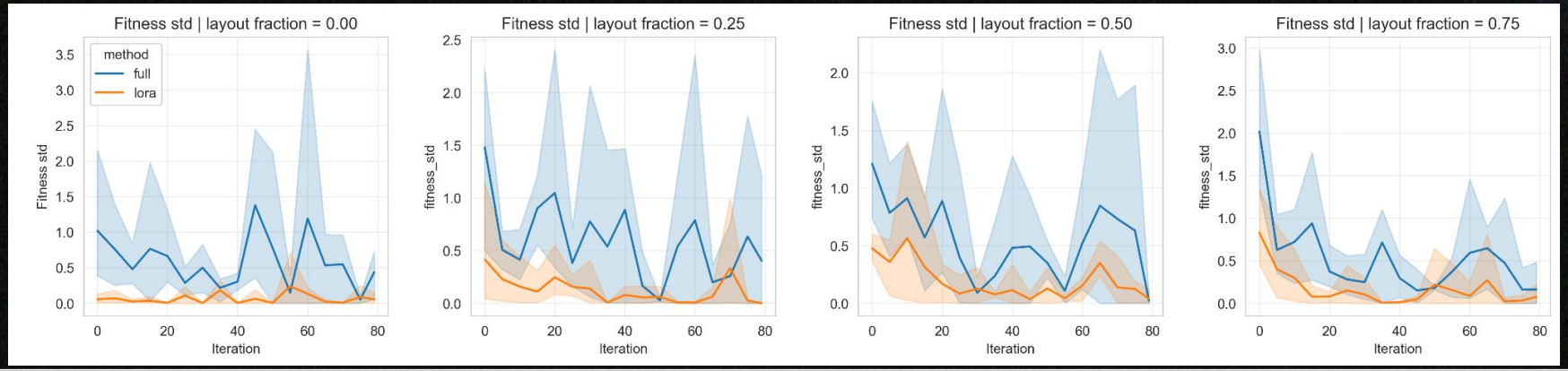


Curriculum Learning + 1-Rank LoRA

ES works well with a warm start, and learning the first environment is most of the work.

So after the first ES training, we only ever update with LoRA after.





Next Steps

- Tune curriculum learning (slight vs. drastic env. perturbations)
- Quantify generalization gap (seen vs unseen envs.)
- Introduce notion of curriculum learning to Wordle
- Then, tune LoRA on Wordle for better curriculum learning